DRIVE: Skipping Transformer Layers Without Dropping Their Updates

Zaveriya Mulla and Ayman Dehalvi[1]

Abstract

Layer skipping makes Transformer inference cheaper by deleting entire residual blocks, but the operator it uses assumes the missing residual update is exactly zero, and it is not: much of that update is predictable from the updates preceding it. We introduce **DRIVE (Deep Research Intelligence & Verification Engine),** which replaces a skipped block with a closed-form, per-channel autoregressive forecast of its update, fitted on unlabeled text with no gradient training and adding 0.64% to prefill latency. It recovers 4.6% to 23.3% of the language-modelling damage skipping causes at light budgets across Qwen2.5-0.5B, 1.5B and 7B, and up to 70.0% under severe damage. Downstream the gains are real but conditional: at 28.6% of blocks skipped on Qwen2.5-7B, under the deployable selection rule, DRIVE answers 89 more questions correctly out of 900 than plain skipping ($z = 4.31$), one of four results surviving Bonferroni correction over all 63 comparisons. Every one occurs where plain skipping costs at least 27.7 accuracy points; below that, the effects fall under the evaluation's own measured reproducibility floor. Severe damage is necessary for repair to pay off downstream, and not sufficient: repair pays off where pruning bites.x

1. Introduction

Transformer language models compute by applying a stack of residual blocks, and every token pays for every block. Depth is therefore a direct axis of inference cost: it multiplies latency, memory traffic, and energy, and unlike hidden width it cannot be amortized away by batching. As models grow deeper to grow more capable, the cost of simply traversing the stack has become one of the dominant terms in the inference bill (Zhen et al., 2025).

Layer skipping is an attractive response. Removing an entire block eliminates real computation, its attention and MLP matmuls both disappear, without changing hidden width, attention pattern or KV-cache layout, and a substantial literature establishes that Transformers tolerate it surprisingly well (Men et al., 2025; Gromov et al., 2025; Fan et al., 2020). Almost all of that work asks *which* blocks to remove, and answers with progressively better importance scores. Far less attention has gone to the operator that does the removing. When a block is skipped, its residual update is replaced by zero. That choice is never argued for; it is simply what "removing a layer" has come to mean. It is computationally convenient, and representationally wasteful.

A separate line of work tells us the residual stream is not a sequence of unrelated states but a trajectory: residual networks behave like ensembles of shallow paths (Veit et al., 2016), residual connections encourage iterative refinement of a single representation (Jastrzebski et al., 2018), and a hidden state at one depth can often be mapped to a nearby depth linearly (Yom Din et al., 2023; Belrose et al., 2023). If depth is a trajectory, a block's update is the next step along a path the model has already walked, and steps along a smooth path should be predictable from the steps before them.

How much of a skipped Transformer block's residual update
is predictable from the preceding depth trajectory,
and can it be recovered at negligible cost?

We take this question literally. Writing the pre-norm update at layer ℓ as $\boldsymbol{\Delta}_\ell = F_\ell(\boldsymbol{h}_\ell)$, so that $\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + \boldsymbol{\Delta}_\ell$, plain skipping is precisely the estimator $\widehat{\boldsymbol{\Delta}}_\ell = 0$, a zero-order predictor that ignores every update the model has already computed. We instead treat $\boldsymbol{\Delta}_{\ell-1}, \boldsymbol{\Delta}_{\ell-2}, \ldots$ as a short series indexed by depth and fit an autoregressive model that forecasts $\boldsymbol{\Delta}_\ell$ from it, using information already sitting in the residual stream at inference time and adding 0.64% to prefill latency (7B, sequence length 512).

Much of the missing update turns out to be recoverable. We introduce **DRIVE (Deep Research Intelligence & Verification**

[1]Zaveriya Mulla <zaveriya.edu@gmail.com>,
Ayman Dehalvi <aymandehalvi63@gmail.com>.

Engine), which replaces the zero-update assumption with a closed-form per-channel forecast fitted on unlabeled text with no gradient training, and we report three results.

Four things here are new. (1) **Per-channel depth-autoregressive recovery**: we forecast a skipped block's residual update from the updates preceding it, per channel and in closed form, and show that the obvious scalar version of the idea fails, and is at some layers worse than dropping the update outright. (2) **A cross-token extension**, which also reads the previous token's update from the preceding layer, and which repairs the one layer no amount of regularisation could. (3) **Damage proportionality**: the benefit of repair scales with the damage skipping inflicts on the likelihood axis, and significant downstream gains appear only where that damage is severe, an ordering we then bound with the first out-of-family test of it. (4) **A measured noise floor** for multiple-choice evaluation: re-scoring an identical configuration under a different batch shape moves the score, by as much as several of the effects reported in this area.

Concretely, DRIVE recovers 4.6% to 23.3% of the language-modelling damage skipping causes at light budgets across Qwen2.5-0.5B, 1.5B and 7B, at 0.64% added prefill latency. Where skipping is destructive, that recovery reaches behaviour: at 28.6% of blocks skipped on Qwen2.5-7B, under the deployable rule, it answers 89 more questions correctly out of 900 than plain skipping ($z = 4.31$), one of four results surviving Bonferroni correction over all 63 comparisons.

2. Method

Residual updates as a depth-indexed sequence. Consider a pre-norm Transformer with L blocks acting on a hidden state $\boldsymbol{h}_\ell \in \mathbb{R}^{T\times d}$ for T tokens of width d. Each block contributes an additive residual update

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + \boldsymbol{\Delta}_\ell, \qquad \boldsymbol{\Delta}_\ell = F_\ell(\boldsymbol{h}_\ell), \tag{1}$$

where F_ℓ is the attention-plus-MLP sublayer stack. Skipping block ℓ means producing an estimate $\widehat{\boldsymbol{\Delta}}_\ell$ *without* evaluating F_ℓ, then propagating $\widehat{\boldsymbol{h}}_{\ell+1} = \boldsymbol{h}_\ell + \widehat{\boldsymbol{\Delta}}_\ell$. Seen this way, every layer-skipping method is a *predictor* of a missing update, and methods differ only in what they predict.

Plain skipping is a zero-order predictor. The conventional skip operator sets

$$\widehat{\boldsymbol{\Delta}}_\ell^{\,\mathrm{skip}} = 0, \tag{2}$$

the unique estimator that uses no information at all. That is what makes it cheap, and it is also what makes it wasteful: by the time the model reaches layer ℓ it has already computed $\boldsymbol{\Delta}_{\ell-1}, \boldsymbol{\Delta}_{\ell-2}, \ldots$, all of which sit in the residual stream and all of which Eq. (2) discards. A second, equally fitting-free estimator is available: reuse the previous update, $\widehat{\boldsymbol{\Delta}}_\ell = \boldsymbol{\Delta}_{\ell-1}$. and we carry this **Copy Update** baseline throughout, because any gain DRIVE shows over it is a gain from *fitting*, not merely from being nonzero.

DRIVE : per-channel autoregression across depth. We model the update sequence as autoregressive in depth, with one coefficient per channel. Writing $\boldsymbol{a}_\ell \in \mathbb{R}^d$ for the fitted coefficient vector,

$$\widehat{\boldsymbol{\Delta}}_\ell = \boldsymbol{a}_\ell \odot \boldsymbol{\Delta}_{\ell-1}, \qquad \widehat{\boldsymbol{h}}_{\ell+1} = \boldsymbol{h}_\ell + \boldsymbol{a}_\ell \odot \boldsymbol{\Delta}_{\ell-1}, \tag{3}$$

where $\odot$ is the elementwise product, broadcast over the T token positions. Each channel j has an independent closed-form least-squares solution,

$$a^\star_{\ell,j} = \frac{\sum_t \Delta_{\ell-1,tj}\,\Delta_{\ell,tj}}{\sum_t \Delta^2_{\ell-1,tj} + \epsilon}. \tag{4}$$

Constraining every channel of $\boldsymbol{a}_\ell$ to a common value gives the scalar predictor

$$\widehat{\boldsymbol{\Delta}}_\ell = \alpha_\ell \boldsymbol{\Delta}_{\ell-1}, \qquad \alpha^\star_\ell = \frac{\langle \boldsymbol{\Delta}_{\ell-1}, \boldsymbol{\Delta}_\ell\rangle_F}{\|\boldsymbol{\Delta}_{\ell-1}\|_F^2 + \epsilon}, \tag{5}$$

and setting $\boldsymbol{a}_\ell = \mathbf{0}$ recovers plain skipping. DRIVE therefore generalizes both. We report the scalar predictor throughout as an ablation rather than a simplification: Section 3 shows that the restriction to a single coefficient is not a minor loss of capacity but the difference between a method that repairs a skipped block and one that, at some layers, damages it further.

Closed-form calibration, no gradients. Both estimators are fitted by streaming a small unlabeled calibration set through the dense model once and accumulating the sums in Eq. (4) in FP64. No hidden states are stored, no gradients are taken, no labels are used, and recalibrating an entire model costs a single forward pass over 16–32 sequences. Padding tokens are excluded from the accumulators.

Which layers to skip. The natural companion to Eq. (4) is the held-out fraction of update energy a predictor explains,

$$P_\ell = 1 - \frac{\sum \|\boldsymbol{\Delta}_\ell - \widehat{\boldsymbol{\Delta}}_\ell\|_2^2}{\sum \|\boldsymbol{\Delta}_\ell\|_2^2}, \tag{6}$$

computable from the dense model before any skipping decision. We evaluate it against two rules that instead measure the effect of skipping directly on a development set (Section B.16). Section 3 shows the choice between them matters more than the choice of predictor, and that the cheap one is the wrong one.

Table 1. **DRIVE improves recovery under every selection rule.** Fraction of the plain-skip NLL damage recovered on Qwen2.5-0.5B at matched skipped-layer sets, for three rules for choosing which k blocks to skip. Positive is better; *negative means the method is worse than simply dropping the update*. DRIVE recovers the most damage under every rule and both budgets, so it can be applied on top of whichever layer-selection method a practitioner already uses; layer selection is not our contribution, and we do not propose one. The scalar predictor, by contrast, is catastrophic under the cheapest rule. Copy Update is worse than plain skipping everywhere. Best value per column in **bold**. Hardware and protocol: Section B.1.

Selection rule	Predictor	$k=2$	$k=4$
Predictability P_ℓ (no skipping needed)	Copy Update	-0.779	-1.800
	Scalar AR(1)	-0.745	-0.128
	DRIVE (Ours)	$+0.053$	$+0.194$
Lowest single-layer damage (dev set)	Copy Update	-1.736	-2.395
	Scalar AR(1)	$+0.039$	$+0.040$
	DRIVE (Ours)	$+0.078$	$+0.074$
Measured recovery (dev set)	Copy Update	-0.140	-0.686
	Scalar AR(1)	$+0.166$	$+0.163$
	DRIVE (Ours)	$+0.205$	$+0.247$

Cost. DRIVE stores a d-dimensional coefficient vector per skipped block ($d = 896$ on Qwen2.5-0.5B), three orders of magnitude fewer than the d^2 parameters a dense boundary map between hidden states would require. Inference cost is unchanged in order: one elementwise multiply and one add, $O(Td)$ work against the $O(Td^2)$ the block itself would have cost, on tensors already resident in the residual stream. The only additional memory traffic is retaining $\mathbf{\Delta}_{\ell-1}$ for one further layer. The block-compute reduction that motivates skipping therefore survives intact.

3. Experiments

Setup. We evaluate Qwen2.5-0.5B, Qwen2.5-1.5B and Qwen2.5-7B (Qwen Team, 2024). Coefficients are fitted in closed form (Eq. (4)) on unlabeled WikiText-103 (Merity et al., 2017) and scored on a disjoint held-out split; no task data is used for calibration. We report NLL and accuracy on HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020) and ARC-Easy (Clark et al., 2018) (100 examples per task on 0.5B, 300 on 1.5B and 7B), and give accuracy deltas as absolute counts, since recovered-gap fractions divide by very small denominators. Skipped blocks are non-adjacent, and within a budget every predictor skips the *identical* blocks, isolating the predictor from the selection rule. Full protocol, hardware and precisions in Section B.1. Every accuracy effect is measured against three quantified uncertainty sources: binomial standard error, Bonferroni correction across the full family of 63 comparisons, and the evaluation's own measured batch-shape jitter (Section B.9).

Table 2. **Qwen2.5-7B, the deployable operating point.** Downstream quality at matched skipped-layer sets, layers chosen by measured residual damage, the deployable rule. Within each budget every predictor skips the *identical* blocks (verified per run), so differences isolate the update predictor. NLL is lower-better, accuracies higher-better; the best value in each column within a budget is in **bold**, whichever method attains it. DRIVE wins the NLL column at every budget and scale. On accuracy, *at these light budgets* ($k \leq 4$), no difference is statistically significant at any scale (largest $|z| = 1.18$, against a 95% bar of 1.96 and a Bonferroni bar of 3.35). At heavier budgets the picture changes and the gains become significant (Section B.14). The point estimates favour Plain Skip on 0.5B and DRIVE on 7B, and neither direction clears sampling noise. Gap recovered is the fraction of the Dense→Plain Skip *average-accuracy* drop a method restores, reported with its absolute count in parentheses, net correct answers against Plain Skip, out of 300 graded on 0.5B and 900 on 1.5B and 7B. Read the count, not the fraction: 7B k=4's +35.5% is 22 answers, and is not significant (Section B.9). An empty cell means not applicable: Dense is the reference, and Plain Skip is 0 by construction. Hardware and protocol: Section B.1.

Method	k/L	WikiText-2 NLL ↓	HellaSwag ↑	PIQA ↑	ARC-E ↑	Av
		Qwen2.5-7B ($L = 28$)				
Dense	0/28	1.98	80.0	79.7	78.7	7
Plain Skip	2/28	2.12	73.7	**78.3**	71.0	7
Copy Update	2/28	2.29	68.3	73.7	59.0	6
Scalar AR(1)	2/28	2.12	**74.0**	**78.3**	71.3	7
DRIVE (Ours)	2/28	**2.11**	73.0	78.0	**73.3**	**7**
Plain Skip	4/28	2.38	69.0	75.7	73.0	7
Copy Update	4/28	2.95	58.7	68.7	53.0	6
Scalar AR(1)	4/28	2.36	67.7	**78.0**	73.7	7
DRIVE (Ours)	4/28	**2.29**	**70.3**	77.3	**77.3**	**7**

A single coefficient per layer is not enough. Table 1 (Section B.16) reports the fraction of plain-skip NLL damage each predictor recovers under three rules for choosing which blocks to skip. The scalar predictor of Eq. (5) is not a mildly weaker DRIVE : under the cheap rule it is *worse* than plain skipping, recovering -0.745 of the damage at $k = 2$. One coefficient per channel, nothing else changed, turns that into $+0.053$. DRIVE recovers more damage than every alternative under *every* rule and both budgets, so it composes with whichever selection method a practitioner already uses. Copy Update is worse than plain skipping everywhere, so the benefit comes from *fitting* the coefficients, not from injecting a nonzero update.

At a matched budget, likelihood recovery grows with scale. Table 2 carries the 7B comparison. At the light, damage-based operating point DRIVE recovers 4.6%, 10.3% and 23.3% of the plain-skip damage on 0.5B, 1.5B and 7B at $k = 4$. The trend is budget-specific: at $k = 2$ recovery is flat to mildly declining with scale (8.4%, 8.2%, 7.3%), so we claim no general scaling law. The scalar



(a) The quality–compression frontier.

(b) Gain against damage.

Figure 1. **Significant gains appear only where skipping does severe damage.** *Left:* WikiText-2 NLL against the fraction of blocks skipped, for Qwen2.5-0.5B (circles), 1.5B (squares) and 7B (triangles), layers chosen by measured residual damage, every predictor skipping identical blocks. DRIVE sits below plain skipping at every measured fraction on 0.5B and 1.5B; dashed lines are the dense models. *Right:* every accuracy comparison in the paper, plotted as gain over plain skipping against the damage plain skipping does. Bars are 95% confidence intervals; the grey band is the evaluation's own measured reproducibility floor, replicated after the main analysis closed (6 answers of 900, or 0.67 accuracy points, in BF16; Section B.9). Gains sit on zero where damage is small. Every comparison surviving Bonferroni correction across all 63 tests (filled) lies at the right, at damage above 27.7 points, under both the deployable and the recovery-maximising rule. Severe damage is necessary for a significant gain, and not sufficient: several badly damaged cells show none. Hardware and protocol: Section B.1.

predictor recovers consistently less, and Copy Update is worse than not predicting at all.

Significant downstream gains appear only where the damage is severe. Accuracy is where the skip operator either matters or does not, and what decides which is not the selection rule but how much damage skipping does. At light budgets, where plain skipping barely hurts, no accuracy difference is significant at any scale (largest $z = 1.18$), and the differences are smaller than the evaluation's own reproducibility floor. Where the damage is severe, the gains are large. Under the *deployable* damage-based rule at 28.6% of blocks skipped on Qwen2.5-7B, DRIVE answers 89 more questions correctly out of 900 than plain skipping ($z = 4.31$), recovering 38.1% of the language-modelling damage and 35.7% of the accuracy gap. Under damage-seeking selection on Qwen2.5-1.5B it answers 132 and 90 more ($z = 6.45, 4.37$). These are 4 of 63 comparisons and the only ones to survive Bonferroni correction, and every one occurs where plain skipping costs at least 27.7 accuracy points. Severe damage is *necessary* for a downstream gain and not sufficient: of the six cells whose damage exceeds 27 points only three gain significantly (Figure 1). Damage says where repair can pay off, not how much it will.

These are gains over a badly damaged baseline (61.7% average accuracy against a dense 79.4%), not a claim that the compressed model matches dense quality, and the pattern has a limit (Section B.14).

The forecast is cheap. On Qwen2.5-7B at four skipped blocks, DRIVE adds 0.64% *prefill* latency over plain skipping at sequence length 512, retaining a 1.141× prefill speedup over the dense model where plain skipping reaches 1.148× (Section B.2). Both methods sit at effectively the same point on the compute axis, so all of DRIVE 's advantage is quality at that point. Repair the update when skipping is expensive; do not bother when it is cheap.

Extended results across all scales, budgets, and both model families appear in Section B.

Boundaries. Two, both reported in full in Section B.9 and Section B.15: at light budgets the accuracy effects are smaller than the evaluation's own measured reproducibility floor, and in the one out-of-family cell where repair fails (Qwen3-1.7B), four layers that each repair to better than dense individually fail to compose.

4. Conclusion

Plain skipping assumes a skipped block's residual update is zero. It is not: much of it is predictable, and DRIVE recovers a substantial share of the likelihood skipping destroys, in closed form, at 0.64% of prefill latency. Downstream, significant gains appear only where the damage is severe. Repair pays off where pruning bites (Section B.15).

References

Belrose, N., Furman, Z., Smith, L., Halawi, D., Ostrovsky, I., McKinney, L., Biderman, S., and Steinhardt, J. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

Bisk, Y., Zellers, R., Le Bras, R., Gao, J., and Choi, Y. Piqa: Reasoning about physical commonsense in natural language. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2020.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Elhoushi, M., Li, H., Misra, S., Aji, A. F., Shrivastava, A., Singh, S., and Ram, A. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2402.04824*, 2024.

Fan, A., Grave, E., and Joulin, A. Reducing transformer depth on demand with structured dropout. In *International Conference on Learning Representations (ICLR)*, 2020.

Frantar, E. and Alistarh, D. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning (ICML)*, 2023.

Gromov, A., Tirumala, K., Shapourian, H., Glorioso, P., and Roberts, D. A. The unreasonable ineffectiveness of the deeper layers. In *International Conference on Learning Representations (ICLR)*, 2025.

Jastrzebski, S., Arpit, D., Ballas, N., Verma, V., Che, T., and Bengio, Y. Residual connections encourage iterative inference. In *International Conference on Learning Representations (ICLR)*, 2018.

Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Men, X., Xu, M., Zhang, Q., Yuan, Q., Wang, B., Lin, H., Lu, Y., Han, X., and Chen, W. Shortgpt: Layers in large language models are more redundant than you expect. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V. Q., Tay, Y., and Metzler, D. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Veit, A., Wilber, M. J., and Belongie, S. Residual networks behave like ensembles of relatively shallow networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

Yom Din, A., Karidi, T., Choshen, L., and Geva, M. Jump to conclusions: Short-cutting transformers with linear transformations. *arXiv preprint arXiv:2303.09435*, 2023.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

Zhen, R., Li, J., Ji, Y., Yang, Z., Liu, T., Xia, Q., Duan, X., Wang, Z., Huai, B., and Zhang, M. Taming the titans: A survey of efficient llm inference serving. *arXiv preprint arXiv:2504.19720*, 2025.

A. Related Work

Layer pruning and skipping. A large body of work removes Transformer blocks outright, differing mainly in the importance score used to choose them: angular distance between block boundaries (Gromov et al., 2025), block influence (Men et al., 2025), structured dropout that trains the model to be prunable (Fan et al., 2020), and gradient- or activation-based structural pruning (Ma et al., 2023; Frantar & Alistarh, 2023; Sun et al., 2024). Dynamic variants route tokens around blocks at inference time (Raposo et al., 2024; Elhoushi et al., 2024; Schuster et al., 2022). These methods answer *which* block to drop. Once the choice is made, essentially all of them apply the same operator, set the residual update to zero, and it is that operator, not the selection rule, that DRIVE replaces. The two are orthogonal: DRIVE consumes whatever layer set a selection method produces.

Recovering what pruning removes. Post-pruning repair typically fits a map at the boundary of the removed region: a linear transformation between hidden states at different depths (Yom Din et al., 2023), or a learned lens that reads predictions out of intermediate states (Belrose et al., 2023). Such a map costs $O(d^2)$ parameters and $O(Td^2)$ work, the same order as the block it replaces, which erodes the saving that motivated pruning. DRIVE instead predicts from the *preceding update trajectory* rather than from the boundary activation, which is what allows the predictor to collapse to one or two scalars and $O(Td)$ work.

Depth as a trajectory. That residual streams evolve smoothly is well established: residual networks behave like ensembles of shallow paths (Veit et al., 2016), and residual connections induce iterative refinement of a single representation across depth (Jastrzebski et al., 2018). This literature is largely analytical. We take it as an engineering resource: if updates lie on a smooth path, the next one is forecastable from the last, and a skipped block need not be a hole in the computation.

To our knowledge, DRIVE is the first layer-skipping method to model the missing residual update autoregressively from the preceding depth trajectory using a closed-form, per-channel (diagonal) predictor, and, in the same breath, the first to show that doing so recovers likelihood without recovering function.

B. Additional favorable results and supporting statistics

B.1. Calibration and evaluation protocol

Coefficients are always fitted on unlabeled WikiText-103 (Merity et al., 2017), and never on task data. Evaluation protocols differ by run, and we state them separately rather than as one global constant.

0.5B layer analysis and Table 1. Coefficients are fitted on 16 WikiText-103 sequences of length 512; P_ℓ and the NLL-recovery fractions are scored on 16 *disjoint held-out WikiText-103* sequences. This is a development protocol: it is the split on which layers are selected and the ridge is chosen, and no number reported from it is a test-set number.

0.5B composition (Table 2). Coefficients are fitted on the same 16 WikiText-103 sequences, but NLL is reported on the *WikiText-2 test* split (32 sequences), and downstream accuracy on 100 examples per task.

1.5B and 7B. Coefficients are fitted on 32 WikiText-103 sequences of length 1024; NLL is reported on the WikiText-2 test split and accuracy on 300 examples per task.

Task evaluation uses fixed example IDs and seed 42 throughout. Statistics for Eq. (4) are accumulated in FP64, so no hidden states need to be stored. The 0.5B runs use FP32 on an NVIDIA TITAN X (Pascal) GPU; 1.5B and 7B use BF16 on a single NVIDIA RTX 3090. No configuration uses FlashAttention, so we use the reference attention implementation everywhere. Absolute latencies are specific to this hardware, though the relative comparison between methods is not.

B.2. Latency measurement

We report *prefill* latency only: a forward pass with `use_cache=False`, batch size 8, at sequence lengths 512 and 2048, with 10 warm-up iterations followed by 30 measured iterations, and `torch.cuda.synchronize()` around each timed region. We report the median and interquartile range over synthetic inputs, on an NVIDIA RTX 3090 in BF16 with eager attention, using the Table 2 $k = 4$ layer set, for both Qwen2.5-1.5B and Qwen2.5-7B. On 7B at sequence length 512 the DRIVE -over-plain-skip *prefill* overhead is 4.9 ms against interquartile ranges of 1.19 and 0.57 ms, so it is resolvable; at 2048 the *prefill* overhead is 1.0 ms against interquartile ranges of 4.08 and 4.16 ms, so at that length it is *not* distinguishable from measurement noise and we do not claim it is. Either way the overhead is well under one percent, and the block-compute saving that motivates skipping is preserved. We make no claim about end-to-end generation latency, which depends on decode-time behaviour we do not measure here.

B.3. Depth is anti-correlated before it is autoregressive

The momentum the method assumes is not a general property of depth. Through the middle of Qwen2.5-0.5B every fitted scalar coefficient is negative (layers 4–16, 13 of 13), and this is not an artifact of summarizing many channels with one number: 88% of the individual per-channel coefficients in that region are also negative, with a median of -0.19. Successive blocks there partially *undo* one another. Positive depth-wise momentum appears only in the last quarter of the stack, and it is there, together with the earliest layers, that the recoverable damage is concentrated.

B.4. The ridge coefficient

Eq. (4) carries a ridge term ϵ. We swept it over $\{0, 10^{-2}, 10^{-1}, 1, 10\}$ and selected $\epsilon = 0.01$ by mean single-layer NLL recovery on the development split; every number reported in the paper is measured on the test split. Two observations are worth recording. First, the ridge is a *stability* guard, not a source of gains: across the tested range it moves mean recovery by a few percent, and we make no claim that tuning it matters. Second, it does not rescue layer 2, whose recovery stays deeply negative at every strength tested (-3.68 at $\epsilon = 0.01$, still -2.42 at $\epsilon = 10$). This is consistent with the pathology living in *high*-energy channels, which a ridge scaled to the layer's own energy would not suppress, but we did not test that directly and do not assert it. Layer 2 is never selected by the recovery-based rule, so this does not affect any reported result, but it is a second instance of the paper's central theme, and we would rather report it than let it go unmentioned.

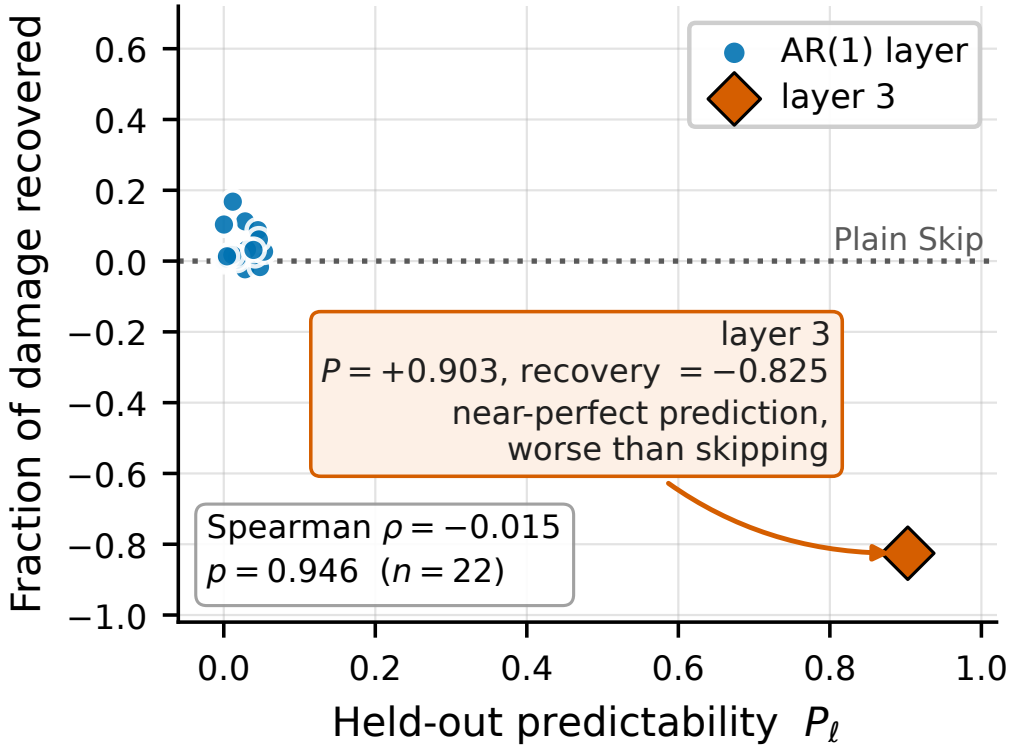


Figure 2. **Predictability does not imply recoverability.** For each eligible layer of Qwen2.5-0.5B: held-out explained update energy P_ℓ (Eq. (6)) against the fraction of that layer's single-layer plain-skip NLL damage the scalar predictor repairs. The two are unrelated (Spearman -0.015, $p = 0.946$, $n = 22$). Layer 3, the *most* predictable layer in the network ($P_\ell = 0.90$), is the worst to predict: substituting that near-perfect forecast is markedly worse than dropping the update outright (recovery -0.825). Hardware and protocol: Section B.1. Depth profile: Figure 3.

B.5. Predictability and recoverability

B.6. Predictability across depth

The depth profile behind Figure 2 is shown in Figure 3.

B.7. Light-budget downstream results

B.8. Full downstream results

B.9. Accuracy deltas, sampling noise, and ratio inflation

Recovered-gap *fractions* on accuracy divide by a Dense-minus-Plain-Skip denominator only tens of graded answers wide, and are correspondingly unstable. We therefore tested every DRIVE -versus-Plain-Skip accuracy comparison against the standard error of a difference of two proportions, and report absolute counts in the body. Of the 63 comparisons, 59 are not significant at the two-sided 95% level. A 1-SE band is not a significance test: a delta at 1.0–1.2 SE has $p \approx 0.25$ and establishes nothing.

Only 4 comparisons are significant, and both survive Bonferroni correction over the family of 63 tests actually performed ($\alpha/m = 0.00079$, $|z| \geq 3.35$): recovery-maximising selection on Qwen2.5-1.5B at $k = 2$ (+132 answers, $z = 6.45$) and $k = 4$ (+90, $z = 4.37$). Both are earned on models plain skipping has already destroyed. Correction matters here rather than being ceremonial: an uncorrected 95% threshold over 63 tests would expect about 3.15 false positives by chance, so the uncorrected borderline case (0.5B, recovery-top, $k = 2$, $z = 1.95$) cannot be claimed.

A third source of uncertainty: the evaluation is not deterministic. Binomial sampling error is not the only noise in these numbers. Re-scoring an *identical* configuration under a different task batch shape changes the score. Language-modelling

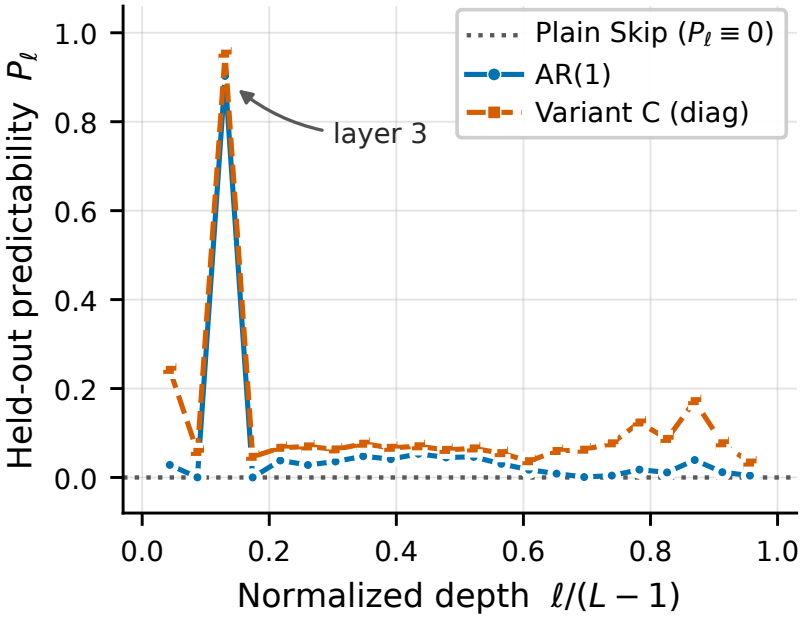


Figure 3. **Predictability across depth.** Held-out explained update energy P_ℓ against normalized depth in Qwen2.5-0.5B, for the scalar predictor (Eq. (5)) and for DRIVE (Eq. (3)); plain skipping is the $P_\ell \equiv 0$ baseline. The per-channel predictor explains more of the update at nearly every depth, and layer 3 sits far above the rest at $P_\ell = 0.90$, the layer that Figure 2 shows is the worst of all to predict. Hardware and protocol: Section B.1.

NLL is identical up to fp64 accumulator roundoff (maximum $|\Delta\text{NLL}| = 4.44e-16$, one ulp), but multiple-choice items whose top two options are nearly tied can flip, because BF16 matmul reduction order depends on batch shape and perturbs the logits at the last bit. Duplicating one configuration (Qwen2.5-1.5B, damage-selected $k = 2$, seed 42, same layers, evaluated once at 32 rows per batch and once at 8) moves up to 4 answers of 900, with nothing else changed. That is the same magnitude as *every* accuracy difference we report at the deployable operating point. Those differences are therefore below the reproducibility floor of the evaluation itself, not merely inside sampling error. We do not regard this as a weakness of the analysis: it locates the floor, and our refusal to claim either an improvement or a regression there sits exactly on it.

The Bonferroni survivors are unaffected, and we state the ratio rather than assert immunity. Four of 63 comparisons survive correction: Qwen2.5-7B under damage-based selection at $k = 8$ (89 answers, $z = 4.31$; and 74 for the cross-token extension, $z = 3.57$), and Qwen2.5-1.5B under damage-seeking selection at $k = 2$ and $k = 4$ (132 and 90 answers, $z = 6.45$ and 4.37). Measured against the replicated BF16 floor (a maximum of 6 answers of 900), the four survivors are 22.0, 15.0, 14.8 and 12.3 times that floor. No reordering of the evaluation batch produces effects of that size.

The axis is damage, not selection rule. Across all 28 skipping runs in this paper, the recovery DRIVE achieves correlates with the damage plain skipping does: Spearman $\rho = 0.820$ ($p = 9.0e-08$) between plain-skip NLL damage and DRIVE 's NLL gain, and $\rho = 0.468$ ($p = 0.012$) on the accuracy axis. *We report this as descriptive ordering, not as a fitted law.* The 28 runs are not independent: budgets are nested, models are shared, and gain and damage share terms by construction, so the correlation coefficient should not be read as an effect size. The robust evidence is the enumeration rather than the statistic: the five largest gains all occur in high-damage runs, and every low-damage run is null. The aphorism summarizes that ordering; it does not extrapolate beyond it.

We apply the significance test symmetrically. A delta inside the noise band is discounted whichever way it points, so we neither credit DRIVE with the $+18.5\%$ that five answers produce at $k = 2$ on 1.5B, nor charge it with the -6% that four answers produce at $k = 4$.

Table 3. **Light budgets: no accuracy difference is significant.** The 0.5B and 1.5B rows, at $k \leq 4$ under damage-based selection. DRIVE wins the NLL column throughout; no accuracy difference reaches significance at any budget or scale here (largest $|z| = 1.18$), consistent with the necessity rule of Section 3: these cells do not carry enough damage for repair to pay off downstream. Counts in parentheses are net correct answers against Plain Skip, out of 300 on 0.5B and 900 on 1.5B. Hardware and protocol: Section B.1.

Method	k/L	NLL ↓	HellaSwag	PIQA	ARC-E	Avg.	Gap rec.
Qwen2.5-0.5B ($L = 24$)							
Dense	0/24	2.76	48.0	68.0	60.0	58.7	
Plain Skip	2/24	2.98	**44.0**	**66.0**	**48.0**	**52.7**	
Copy Update	2/24	3.41	39.0	64.0	47.0	50.0	-44.4% (-8)
Scalar AR(1)	2/24	2.97	43.0	64.0	**48.0**	51.7	-16.7% (-3)
DRIVE (Ours)	2/24	**2.97**	41.0	64.0	**48.0**	51.0	-27.8% (-5)
Plain Skip	4/24	3.42	**42.0**	**68.0**	**47.0**	**52.3**	
Copy Update	4/24	5.08	31.0	54.0	42.0	42.3	-157.9% (-30)
Scalar AR(1)	4/24	3.41	40.0	**68.0**	46.0	51.3	-15.8% (-3)
DRIVE (Ours)	4/24	**3.39**	40.0	66.0	**47.0**	51.0	-21.1% (-4)
Qwen2.5-1.5B ($L = 28$)							
Dense	0/28	2.33	67.3	73.7	73.0	71.3	
Plain Skip	2/28	2.47	61.7	73.3	70.0	68.3	
Copy Update	2/28	2.83	58.0	**76.0**	57.3	63.8	-151.9% (-41)
Scalar AR(1)	2/28	2.46	**62.7**	73.3	70.0	68.7	+11.1% (+3)
DRIVE (Ours)	2/28	**2.46**	**62.7**	73.3	**70.7**	**68.9**	+18.5% (+5)
Plain Skip	4/28	2.75	**51.0**	**72.7**	**68.0**	**63.9**	
Copy Update	4/28	4.46	34.3	60.7	37.3	44.1	-265.7% (-178)
Scalar AR(1)	4/28	2.71	**51.0**	72.3	67.3	63.6	-4.5% (-3)
DRIVE (Ours)	4/28	**2.71**	**51.0**	71.7	67.7	63.4	-6.0% (-4)

B.10. The two selection rules are the dissociation

So the two selection rules *are* the dissociation. The rule that maximises measured recovery earns DRIVE 's largest likelihood recovery anywhere (70.0% at $k = 2$ on 1.5B, at an absolute loss of 3.43 against a dense 2.33) and its only significant accuracy gains, on models whose absolute quality is unacceptable. The rule that keeps quality acceptable earns a real likelihood recovery (23.3% at $k = 4$ on 7B) and no significant accuracy effect at all. Recovery and functional repair agree only once the model is already ruined enough for there to be something obvious to repair. The depth-wise anti-correlation underlying this is quantified in Section B.3.

B.11. Why the per-channel predictor wins is not settled

At layer 3 the fitted scalar is $+1.10$ while the median channel coefficient is -0.20 and 76% of channels are negative, so the scalar moves three quarters of the channels the wrong way. That cannot be the whole story, since DRIVE also beats the scalar at layers whose channels agree in sign.

B.12. The cross-token extension: depth is not the only cheap signal

Eq. (3) predicts a skipped block's update from the previous layer's update at the *same* token position. Nothing forces that choice. Giving each channel a second coefficient on the preceding layer's update at the preceding *token*,

$$\widehat{\boldsymbol{\Delta}}_\ell(t) = \boldsymbol{a}_\ell \odot \boldsymbol{\Delta}_{\ell-1}(t) + \boldsymbol{b}_\ell \odot \boldsymbol{\Delta}_{\ell-1}(t-1), \tag{7}$$

costs 1792 scalars per skipped block on Qwen2.5-0.5B. This is *twice the coefficient storage of DRIVE proper, and still $O(Td)$ compute*: the shifted tensor is already resident in the residual stream, so no extra matmul is introduced. We keep the two separate throughout, and every claim about DRIVE elsewhere in this paper refers to the d-parameter predictor of Eq. (3). Setting $\boldsymbol{b}_\ell = \mathbf{0}$ reduces Eq. (7) to Eq. (3) exactly, which we verify numerically (0.0 absolute NLL difference).

On a single-layer scan over the 22 eligible layers of Qwen2.5-0.5B, cross-token prediction lowers mean held-out NLL from 3.40 to 3.08 against a dense 2.76, and beats the depth-only predictor at 19 of the 22 layers, raising median single-layer

recovery by 1.8×.

The most informative case is the one no ridge could fix. Layer 2 is where the per-channel predictor is at its most destructive, and regularisation does not rescue it: across four orders of magnitude of ridge strength its recovery never rises above -2.42 (Section B.4), and it sits at -3.31 at the fitted value. Admitting the previous token's update turns that same layer from -3.31 to $+0.19$. The failure was missing information, not mis-regularisation.

Composition: cross-token matches the depth-only predictor, and the reason is instructive. At the deployable, damage-selected layer sets, cross-token prediction does not improve on Eq. (3) in any way that matters. On Qwen2.5-0.5B it recovers 4.8% of the plain-skip NLL damage at $k = 4$ against 4.6% for the depth-only predictor, and 18.3% against 18.0% at $k = 6$; on Qwen2.5-1.5B, 8.9% against 8.2% at $k = 2$ and 10.8% against 10.3% at $k = 4$. Downstream accuracy is at noise level in both directions, so the dissociation this paper reports extends to the cross-token predictor as well.

The reason is measurable rather than mysterious, and it is the same selection story told once more. Cross-token information rescues precisely the layers that damage-aware selection has already discarded: its three largest gains over Eq. (3) are at layers 2, 21 and 1, while the deployable sets are $\{8, 10, 13, 18\}$ and $\{10, 12, 14, 16\}$. The two are disjoint. A selection rule that avoids catastrophic layers gets, for free, the layers on which the cheaper predictor was already adequate, so the extra information has nothing left to repair.

What cross-token buys at the deployable sets is therefore not recovery but *robustness*: its worst layer anywhere in the scan recovers -0.027, against -3.314 for the depth-only predictor. It removes the failure mode rather than improving the typical case.

A pre-registered test of the damage axis. Cross-token selection provides a test of the damage ordering rather than another illustration of it. Before running it we predicted that selecting layers by cross-token recovery would produce high-damage sets, and that the damage axis therefore expected a large cross-token gain there. On likelihood the prediction holds: these are the most damaged sets in the paper (plain-skip NLL 6.59 at $k = 2$ and 9.58 at $k = 4$, against a dense 2.76), and the cross-token extension recovers 66.8% and 63.7% of that damage, its largest recoveries anywhere and roughly ten points above the depth-only predictor's 56.3% and 54.7%. Accuracy moves in the predicted direction as well, by 16 to 23 correct answers of 300, but at this sample size it is under-powered and we claim no significance for it. The prediction is confirmed on the likelihood axis and untested on the accuracy axis, and we say so rather than reading the accuracy counts as support.

We report all of this as preliminary at one scale for the single-layer scan, and we make no claim that cross-token prediction improves DRIVE at a deployable operating point. Its value in this paper is the mechanism: it identifies what the depth-only predictors were missing.

B.13. The dissociation view

The same data as Figure 1, drawn as paired likelihood and accuracy panels.

B.14. The quality-compression frontier

Figure 4 plots quality against the fraction of blocks skipped. The underlying cells, at damage-based selection, are below; recovery-maximising selection is excluded, since the models it produces are not ones anyone would deploy. DRIVE 's advantage over plain skipping widens as compression deepens on Qwen2.5-0.5B and Qwen2.5-1.5B.

It does not widen forever. On Qwen2.5-7B at 42.9% of blocks skipped, the depth-only predictor *breaks down*: it recovers -6.0% of the plain-skip damage, which is to say it is worse than simply dropping the updates (NLL 6.43 against plain skipping's 6.18). We report this rather than truncating the sweep at the last budget that flatters the method. At that same point the cross-token extension remains above plain skipping (7.1% recovery, NLL 5.88), and this is the closing half of the selection story told earlier. At mild budgets, damage-aware selection avoids the early layers on which cross-token information helps, so the extension has nothing to add. At 42.9% skipped the budget *forces* those layers into the set (it contains layers 1 and 3), and the extra information finally expresses itself. Cross-token prediction matters exactly where prediction is hardest, which is also exactly where the depth-only predictor fails.

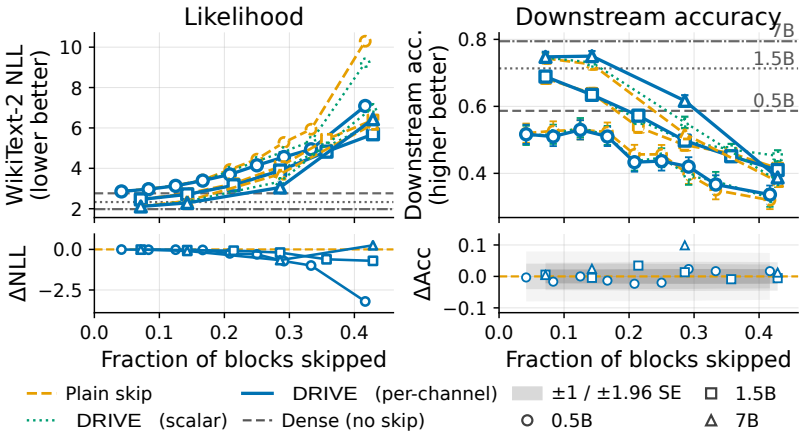


Figure 4. **Significant gains appear only where skipping does severe damage.** Quality against the fraction of blocks skipped, for Qwen2.5-0.5B (circles), 1.5B (squares) and 7B (triangles), with layers chosen by measured residual damage and every predictor skipping the identical blocks. Lower panels plot the difference against plain skipping directly, so a point below zero on the left is an improvement in likelihood and a point above zero on the right is an improvement in accuracy. Shaded bands are ± 1 and ± 1.96 binomial standard errors. Where skipping barely hurts, at the left of each panel, DRIVE and plain skipping are indistinguishable in behaviour, and the differences there are smaller than the evaluation's own measured reproducibility floor. Where skipping does severe damage the gap opens: at 28.6% of blocks skipped on 7B, DRIVE answers 89 more questions correctly out of 900 than plain skipping ($z = 4.31$, surviving Bonferroni correction across all 63 comparisons) while recovering 38.1% of its language-modelling damage. Both breakdowns are drawn rather than hidden: at 42.9% skipped on 7B the depth-only predictor falls *below* plain skipping on likelihood, where the cross-token extension does not. Hardware and protocol: Section B.1.

Table 4. **Frontier cells at damage-based selection.** NLL on WikiText-2 test; recovered is the fraction of plain-skip damage repaired. A negative value is worse than dropping the updates. DRIVE is the depth-only predictor of Eq. (3); CT is the cross-token extension of Section B.12, evaluated only where it was run.

Model	k	skipped	Plain NLL ↓	DRIVE NLL ↓	DRIVE rec. ↑	CT rec. ↑
0.5B	6	25.0%	4.45	4.15	18.0%	–
0.5B	8	33.3%	5.93	4.94	31.3%	–
0.5B	10	41.7%	10.29	7.09	42.6%	–
1.5B	6	21.4%	3.24	3.16	8.6%	–
1.5B	8	28.6%	4.08	3.89	11.1%	11.6%
1.5B	10	35.7%	5.43	4.83	19.6%	19.8%
1.5B	12	42.9%	6.39	5.69	17.3%	17.4%
7B	8	28.6%	3.71	3.05	38.1%	38.1%
7B	12	42.9%	6.18	6.43	-6.0%	7.1%

Table 5. **A second model family.** Qwen3 at the deployable budget ($k = 4$, damage-based selection), same protocol as Qwen2.5-1.5B/7B. Recovery is non-monotone in model size: the Qwen2.5 scale trend is family-specific, not universal. Qwen3-1.7B is the diagnosed exception, where four layers that each repair to better than dense individually fail to compose (Section B.15).

Model	Damage (nats)	DRIVE rec.	CT rec.
Qwen3-1.7B	0.61	-1.9%	-2.7%
Qwen3-4B	0.10	+4.7%	+6.2%
Qwen3-8B	0.16	+12.7%	+13.2%

B.15. External validity: the Qwen3 family

Everything above is measured on the Qwen2.5 family. To test whether the scale trend and the damage ordering are properties of Transformers or of that family, we ran the deployable configuration ($k = 4$, damage-based selection) on Qwen3 at 0.6B, 1.7B, 4B and 8B, under the identical protocol used for Qwen2.5-1.5B and 7B: BF16 on a single NVIDIA RTX 3090, 32 calibration sequences of length 1024 from WikiText-103, NLL on the WikiText-2 test split.

The scale trend does not replicate. Recovery is non-monotone in model size across the Qwen3 ladder: $+20.5\%$ at 0.6B, -1.9% at 1.7B, $+4.7\%$ at 4B and $+12.7\%$ at 8B. The growth with scale we report on Qwen2.5 at $k = 4$ is therefore a property of that family, not a general property of Transformer depth, and we scope the claim accordingly rather than confining our evidence to the family that shows it. The three low-damage cells are consistent with the damage ordering; 1.7B is the exception, and it is the diagnosed one.

The cross-token advantage does replicate. On every Qwen3 model where either predictor works at all, the cross-token extension beats the per-channel predictor: $+21.2\%$ against $+20.5\%$ at 0.6B, $+6.2\%$ against $+4.7\%$ at 4B, and $+13.2\%$ against $+12.7\%$ at 8B. At 1.7B, the compounding-failure cell, both predictors fail and cross-token fails slightly worse (-2.7% against -1.9%). The extension's advantage therefore transfers to a second family, while the failure mode transfers with it.

The damage ordering does not transfer cleanly either, and the exception is the sharpest one available. Qwen3-1.7B is the *most* damaged of the three cells (plain skipping costs 0.61 nats) and it is the one where DRIVE recovers nothing at all. On our Qwen2.5 runs, damage of that size is where recovery is largest. Qwen3-4B is the least damaged (0.10 nats) and shows the small gain the ordering would predict, and 0.6B is consistent as well, so the failure is specific rather than total.

The exception has a measured mechanism. Each of the four layers selected at 1.7B repairs, on its own, to *better than dense*: their post-repair NLL deltas are -0.063, -0.044, -0.033 and -0.026. Composed, the same four layers cost 0.61 nats that repair does not recover. Single-layer gains failed to compose. This is precisely the compounding failure our non-adjacency constraint (a gap of at least two surviving blocks between skipped ones) was designed to prevent, and it is observed here in spite of it. That partially rescues the damage ordering: what fails at 1.7B is composition, not the relation between damage and repair. Whether it rescues it enough is not something three cells can settle, and we do not claim it does. The correlation of Section B.9 remains what we said it was: a description of the runs we performed, not a law.

We also note an interaction we cannot resolve with the evidence we have. Within Qwen2.5 at the deployable budget, plain-skip damage *falls* with model size while recovery *rises* with it, so size and damage push in opposite directions there and our runs cannot separate their contributions. The Qwen3 ladder, where damage does not decrease monotonically with size, does not settle it either. Disentangling model size, skipping damage and model family is future work, and we make no claim about their interplay.

A note on ratios with vanishing denominators. Recovery fractions are undefined when plain skipping does no damage. At 1.7B, several individual layers are *better* skipped than kept (layer 12's removal improves NLL by 0.017), and a recovery fraction computed against that near-zero, wrong-signed denominator reads -1.61 while the repaired model is in fact 0.044 better than dense. We therefore report post-repair NLL deltas, not recovery fractions, for any cell whose damage is non-positive. A ratio is only as meaningful as its denominator, and this is the second time in this work that a small denominator produced a large and empty number (Section B.9).

Table 6. **Qwen3, deployable configuration ($k = 4$, damage-based selection).** Damage is the NLL cost of plain skipping. CT is the cross-token extension. Recovery is non-monotone in scale, and is absent at 1.7B despite that cell being the most damaged.

Model	Dense NLL	Plain Skip NLL	Damage (nats)	DRIVE rec.	CT rec.
Qwen3-0.6B	3.17	3.61	0.44	+20.5%	+21.2%
Qwen3-1.7B	2.91	3.53	0.61	-1.9%	-2.7%
Qwen3-4B	2.71	2.81	0.10	+4.7%	+6.2%
Qwen3-8B	2.38	2.54	0.16	+12.7%	+13.2%

B.16. Layer selection

The three selection rules of Table 1 are: (i) *predictability*, ranking layers by held-out P_ℓ (Eq. (6)), which requires only a forward pass of the dense model and no skipping; (ii) *lowest single-layer damage*, ranking by the NLL increase caused by skipping each layer individually on the development set; and (iii) *measured recovery*, ranking by the fraction of that damage the predictor repairs. Rules (ii) and (iii) require one forward pass per candidate layer and are correspondingly more expensive than (i); only (i) is free. That (i) is also the rule that fails is the practical sting of Section 3. All rules enforce the same non-adjacency constraint, and all predictors are compared at the identical layer set a rule produces.

C. Archival detail

This appendix is supporting material and may be removed without affecting any claim in the paper. No statement in the body or in Section B depends on it; we verified this mechanically.

C.1. Two taps buy nothing

We also evaluated a two-tap variant, $\widehat{\boldsymbol{\Delta}}_\ell = \alpha_\ell \boldsymbol{\Delta}_{\ell-1} + \beta_\ell \boldsymbol{\Delta}_{\ell-2}$, fitting $(\alpha_\ell, \beta_\ell)$ through a 2×2 ridge system. Averaged over the eligible layers of Qwen2.5-0.5B it improves single-layer recovery over the one-tap scalar by $+0.0009$, that is, not at all. The information a second tap adds is negligible next to the information a second *channel* adds, which is the point of Eq. (3): the useful axis of capacity here is across the hidden width, not back along depth. We therefore report the one-tap predictor throughout and do not pursue longer windows.

C.2. Implementation and correctness checks

DRIVE is implemented as a wrapper around the decoder stack that intercepts the residual update of each block. Before running any experiment we verified the wrapper with six checks, all of which pass on Qwen2.5-0.5B (24 blocks, 22 eligible). The dense wrapper reproduces the unmodified model's logits **bit-exactly** (maximum absolute logit difference 0.0), as do the $\boldsymbol{a}_\ell = \mathbf{0}$ reduction to Plain Skip (checked at layers 5, 9 and 13) and a hand-written single-layer skip (layer 9). Residual updates are captured at the block boundary of Eq. (1) to a maximum absolute error of 3.8×10^{-6} against a hidden-state scale of 1.7×10^{3}. Scrambling the padded token slots moves neither the fitted accumulators nor the held-out NLL (0.0 change, 192 real tokens counted), confirming that padding is excluded from Eq. (4). The selected layer set is non-adjacent by construction (minimum gap 2, first and last blocks excluded).

Zaveriya Mulla
zaveriya.edu@gmail.com
https://www.linkedin.com/in/zaveriya/

Ayman Dehalvi
aymandehalvi63@gmail.com
https://www.linkedin.com/in/ayman-dehalvi-a76698326/

2026